Table of Contents

Problem Analysis and Requirements: **2**

Design and Justification **2**

Multi Language Limitation: 3

Why DistilBERT 3

What Metrics to use 3

Sklearn and Hugging Face Library 3

Implementation **4**

Environment setup 4

Data Loading 4

Data Preprocessing 5

Sentiment label preparation: 6

Model Training & Testing 6

LogisticRegression 6

LogisticRegression Hyperparameter Tuned 7

LinearSVC & MultinomialNB 8

DistilBERT-Multilingual 8

Evaluation and Results **8**

Conclusion and Limitations **11**

Final Recommendation 12

References **13**

Problem Analysis and Requirements:

A large e-commerce company has collected customer reviews for their products. Review text is unstructured, contains informal language, and the dataset is large. A new model needs to be developed to automatically classify them as having 'Positive', 'Negative', or 'Neutral' sentiment. This will help them quickly identify products with issues and gain a better understanding of customer feedback.

We need to make an informed decision so that the right AI approach is chosen for the customer review analysis. The input file is a parquet file which contains the following columns. Please note that the reviews are in three different languages.

- review_id
- product_id
- reviewer_id
- stars
- review_body
- review_title
- language
- product_category

We need to choose the right model, using either a classical ML or a deep learning approach. Train the selected models and perform a comprehensive performance evaluation using the metrics accuracy, precision, recall and F1-score to justify the final decision.

Design and Justification

There are many classical ML approaches such as Logistic Regression, Support Vector Machine (SVM), Naive Bayes, Random Forest, K-Nearest Neighbors (KNN) etc. For this sentiment analysis we will look at three specific models:

- LogisticRegression
- LinearSVC
- Multinomial Naive Bayes (MNB)

These three models are fast and reliable. As per the documentation, MNB works well with text classification tasks like spam detection and sentiment analysis due to its speed and efficiency. We will check how well all these models work for sentiment analysis with informal language. We will compare the metrics to evaluate them at the end.

Along with this we will also look at one deep learning neural network model. Some of the deep learning models are Long Short-Term Memory (LSTM), Gated Recurrent Unit (GRU), Deep Feedforward Neural Network (Multilayer Perceptron - MLP), Recurrent Convolutional Neural Network (RCNN), 1D Convolutional Neural Network (1D-CNN) etc. These are conventional deep learning models where we need to train and test the models. But for this analysis we will use

advanced transformer-based models as these are pre-trained models with advanced capabilities. We will choose DistilBERT-Multilingual as it is capable of handling multilingual customer reviews. At the end we will compare the evaluation metrics of DistilBERT-Multilingual with Classical ML models.

Multi Language Limitation:

For classical ML we will filter the dataset with only English reviews, as these basic models are not capable of handling any language other than English out of the box.

But for DistilBERT-Multilingual we will go with the full dataset which will have English, German and French, as this model is capable of handling it.

Why DistilBERT

Even though we have many traditional Deep learning models LSTM, GRU, or CNN, we have chosen DistilBERT-Multilingual which is a Natural Language Understanding (NLU) model over its superior capabilities, such as:

- Transformer-based models with pre-trained capabilities.
- Multilingual support capable of handling 100+ languages.
- Self-Attention mechanisms for longer review texts.
- Understands grammar, tone, sarcasm and human language structure.
- Capability to do parallel processing to train the model when we have larger datasets.
- Lightweight compared to other BERT models, but retains the performance.
- Excels at analyzing and extracting structure from existing text.

For more details, see: https://huggingface.co/distilbert/distilbert-base-multilingual-cased

What Metrics to use

Evaluating a model is an important step to measure the performance and choose the correct one for our scenario. We will measure it based on four scores:

1. Accuracy - Measures the overall correct predictions out of all predictions made by the model.
2. Precision - Measures how many of the items the model *predicted* as positive were actually correct (focuses on minimizing false alarms).
3. Recall - Measures how many of the *actual* positive items in the dataset the model managed to find (focuses on minimizing missed detections).
4. F1-Score - Combines Precision and Recall into one number. It balances both false positives (incorrectly flagging something) and false negatives (missing something).

Sklearn and Hugging Face Library

As we have chosen which classical ML and deep learning models to use, we now need to decide how to implement them in Python.

For Classical ML we use scikit-learn (sklearn) to implement classical models which is an industry-standard, open-source Python library.

As defined by Scikit-learn developers (n.d.) “Scikit-learn is an open source machine learning library that supports supervised and unsupervised learning. It also provides various tools for model fitting, data preprocessing, model selection, model evaluation, and many other utilities.”

For deep learning, we are going to use the Hugging Face library. It has a popular open-source Transformers library that offers state-of-the-art transformer-based models, which are easy to implement.

Implementation

Under Design and Justification, we have chosen which models we are going to try and which libraries we are using at a high level. Now we will see the implementation setup.

Environment setup

To run the Python code, we need to set up the environment, and there are different ways to do it. For a local machine, we can directly install the latest Python and run the code or choose one of the online environments offered by different companies. There is also a choice of which IDE to use, like Visual Studio Code (VS Code), PyCharm or an online platform. For this development, I have chosen the Colab kernel to run the code and VS Code for building the code.

Environment: **Colab**. Why? We need a lot of GPU power to train the AI model, and Google offers them with different configurations.

IDE: **VS Code**. Why? It’s a personal choice as I am comfortable with it.

Before running the code, please make sure we have the environment set up correctly, with either a GPU or a CPU, and that it is assigned and set up in VS Code. The figure below illustrates that the environment is good.

```
Python: 3.13.16 (main, Oct  1 2026, 15:40:24) [GCC 13.3.0]
Executable: /usr/bin/python3
Runtime is running Date and time: 2026-11-01 20:09:36
```

Data Loading

We have two datasets of product reviews. The train dataset contains ~20,000 reviews and the test dataset contains ~1,500. To load this data into our code, we can use either the Colab sample folder or Google Drive.

We are using Google Drive to save datasets. Make sure the Drive is mounted first and that the files exist before loading the data. The figure below illustrates that the Drive is mounted and both datasets exist.

Note: While mounting the Drive, there might be pop-ups asking you to grant permission.

```
Mounted at /content/drive
Drive mounted: True
File exists:/content/drive/MyDrive/AI/SampleFile/train-product-review.parquet - True
File exists:/content/drive/MyDrive/AI/SampleFile/test-product-review.parquet - True
```

Data Preprocessing

Data preprocessing is a very important step for the classical ML models, as these are based on words and not on the whole review text and sentiment. In preprocessing, we will clean up the data and also apply stemming, lemmatization and tokenization.

- Remove unnecessary spaces
- Remove URLs
- Remove extra spaces
- Remove special characters
- Stemming: Reduce words by removing prefixes and suffixes
- Lemmatization: Reduce words to their proper dictionary root
- Tokenization: Split text into smaller, manageable units (tokens)
- Drop all rows that have empty or NA values using dropna

After processing the data using the above methods, we will combine the token words and store them. While training, we will be using this processed text for the classical ML models. But note that for the deep learning model we will not be doing any of these steps except dropna.

The figure below illustrates how the actual review and processed data look. We can also see the split of the train data according to the ratings.

```
                                          review_body  \
0     Great smell and tasting tea. Love the box too.
1  Love, love, love the designs! Very pretty! But...
2  It's a really good battery. Works well and you...
3             Honestly Didn't hold anything in place
4       Great Product! Arrived on time as specified.

                              review_body_processed
0                       great smell tast tea love box
1      love love love design pretti wear design eras
2  's realli good batteri work well charg lot tim...
3                       honestli n't hold anyth place
4                    great product arriv time specifi
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6736 entries, 0 to 6735
Count of each star rating:{3: 1406, 5: 1353, 1: 1337, 2: 1320, 4: 1320}
```

Sentiment label preparation:

In the source data, the star rating is a number from 1 to 5. For sentiment analysis we will be using *'Positive', 'Negative', or 'Neutral'* sentiment. So we need to convert the ratings as follows: "positive" for stars >= 4, "negative" for stars <= 2 and "neutral" for stars = 3.

```
sentiment
positive    2673
negative    2657
neutral     1406
```

Model Training & Testing

LogisticRegression

Now we have the train set with preprocessed review text and sentiment labels. The first step is to convert the review text into numerical values that the classical ML models can understand and process. The two common techniques are Bag of Words (BoW) and TF-IDF (Term Frequency-Inverse Document Frequency). Here we use TF-IDF, as it is more advanced.

- TF-IDF weights words by importance instead of only counting how many times a word appears (BoW), so very common words are given less weight and the weight of distinctive words is increased.
- TF-IDF supports two-word pairs (bigrams) as well as single words, so more context is captured.

Assign the review text and sentiment label to train and test. Here we will vectorize both the train and test datasets. Since we want to use bigrams (2-grams), define this in the vectorizer parameter.

For the train dataset, we need to fit and transform the text, and for the test dataset, we only need to transform it to avoid data leakage.

Next, we will train the model using LogisticRegression. Make sure class_weight is set to "balanced". This will help the model to give equal importance to all classes during training, as we have less Neutral data.

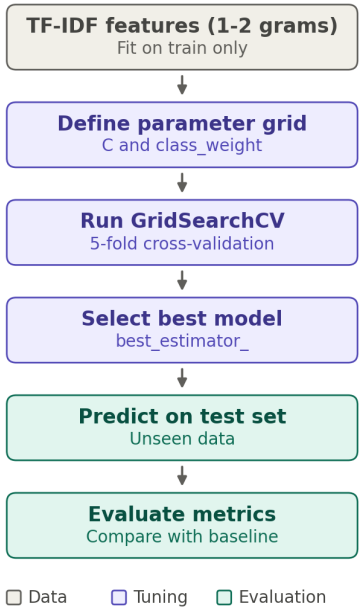
Next, **predict** on the test dataset. Finally, we need to measure the model's performance using the metrics.

LogisticRegression Hyperparameter Tuned

The baseline Logistic Regression used the default C value. Now we can do the hyperparameter tuning and check how the metrics changed. In this step we tune it with GridSearchCV to find the best values for C and class_weight, then check whether tuning improves performance. The best value depends on the dataset, so we find it through cross-validation rather than guessing.

- A smaller C means stronger regularization. This helps prevent overfitting, but if C is too small the model may underfit.
- A larger C means weaker regularization. The model fits the training data more closely, but if C is too large it may overfit.

Vectorizer, Train, Predict and metrics measurement remain the same. The additional step before training is to define a grid of C values, run GridSearchCV, and use the best model for prediction. Also note that we used class_weight="balanced".



LinearSVC & MultinomialNB

For these two models, we will do the same process as for LogisticRegression and collect the metrics.

DistilBERT-Multilingual

The overall process is the same as for the classical ML models, except for the data preprocessing. We removed the manual preprocessing steps (stemming, lemmatization, stopword removal), because the transformer model uses its own tokenizer and understands the full context of longer reviews, so it can predict the overall sentiment. Classical models, in contrast, are word-based and treat words independently.

Hyperparameter tuning is also applied to the DistilBERT-Multilingual model using Optuna. The process has three steps: start the Optuna hyperparameter search, find the best hyperparameters, and apply them for the final training. Instead of going through the whole process, we explain a few important points to look for.

- `fp16=True`. This enables mixed-precision training. Many values are stored in 16-bit instead of 32-bit (FP32) precision, which reduces memory use and speeds up computation on modern GPUs. Without it, memory use can grow quickly and training slows down.
- `dataloader_num_workers.` This controls how many sub-processes load and pre-process data in the background. The next batches are prepared while the GPU is still computing the current batch in the background, so the GPU is not left waiting for data.
- `per_device_eval_batch_size.` This sets how many samples are processed at the same time on each device during evaluation. A larger batch lets more samples be processed in parallel, which makes evaluation faster.
- **per_device_train_batch_size**. This sets how many reviews are processed per training step. Larger batches train faster but need more GPU memory.
- **num_train_epochs.** This sets how many times the model sees the whole training set. Usually 2 to 4 is enough for fine-tuning, and more epochs risk overfitting and waste time.
- **weight_decay**. This adds regularization that discourages overly large weights and helps reduce overfitting.
- **load_best_model_at_end.** This keeps the best checkpoint.

Why are these parameters important? Training a transformer model is a time-consuming and resource-intensive operation. Without these speed-up settings, training could take several hours. Using them reduces training time without significantly compromising model accuracy.

Evaluation and Results

The classical models were trained on 6,736 English reviews (2,673 positive, 2,657 negative and 1,406 neutral) and tested on 460 English reviews.

DistilBERT-Multilingual was fine-tuned on the full multilingual data and tested on 1,500 reviews in German, English and French.

```
Train shape (Rows + columns of words): (6736, 86624)
Test shape (Rows + columns of words): (460, 86624)
Class distribution in train: {'positive': 2673, 'negative': 2657, 'neutral': 1406}
```

The baseline Logistic Regression reached 63.9% accuracy and a weighted F1 of 0.640. The Logistic Regression results show a large gap between the classes. Negative reviews reached an F1-score of 0.71 and Positive reviews 0.73, but the Neutral class scored only 0.30 (precision 0.30, recall 0.30). Since Neutral performance is so low, it pulls the macro F1 down to 0.58, even though overall accuracy is 63.9%.

```
--- Logistic Regression Results ---
Accuracy: 0.6391
              precision    recall  f1-score   support

    negative       0.70      0.73      0.71       188
     neutral       0.30      0.30      0.30        90
    positive       0.76      0.71      0.73       182

    accuracy                           0.64       460
   macro avg       0.58      0.58      0.58       460
weighted avg       0.64      0.64      0.64       460
```

Hyperparameter tuning with GridSearchCV selected C = 1.0 and a balanced class weight, with a best cross-validation F1 of 0.650. The tuned model produced exactly the same test scores as the baseline, which confirms that the default settings were already close to optimal. Using `class_weight="balanced"` and hyperparameter tuning did not improve the Neutral score, which suggests the limitation comes from the features and the data, not from the model settings.

```
Best Parameters Found:
{'C': 1.0, 'class_weight': 'balanced'}
Best Cross-Validation F1-Score: 0.6497

--- Logistic Regression - Hyperparameter Tuning---
              precision    recall  f1-score   support

    negative       0.70      0.73      0.71       188
     neutral       0.30      0.30      0.30        90
    positive       0.76      0.71      0.73       182

    accuracy                           0.64       460
   macro avg       0.58      0.58      0.58       460
weighted avg       0.64      0.64      0.64       460
```

LinearSVC achieved a similar 64.1% accuracy (F1 0.628). Multinomial Naive Bayes had the highest accuracy of the classical models (65.4%) but the lowest precision (0.535) and F1 (0.584), which suggests it favors the larger classes (Positive and Negative).

In theory, Multinomial Naive Bayes (MNB) works well for text classification and sentiment analysis. However, in our evaluation, it did not outperform the other models. It reached the highest accuracy among the classical models (65.4%), but its precision (0.535) and F1-score (0.584) were clearly the lowest, so the accuracy figure is misleading.

A gap this large between accuracy and precision usually means the model is predicting mostly Positive/Negative and performing poorly on Neutral. The model is then right on most Positive and Negative reviews, which keeps accuracy high, but its predictions for the minority class are poor.

All three algorithms finished within about 1.5 points of each other.

```
================================================================
            SIDE-BY-SIDE MODEL PERFORMANCE COMPARISON
================================================================
                                      Accuracy  Precision  Recall  F1-Score
LogisticRegression                      0.6391     0.6413  0.6391    0.6398
LogisticRegression Hyperparameter Tuned 0.6391     0.6413  0.6391    0.6398
LinearSVC                               0.6413     0.6205  0.6413    0.6275
MultinomialNB                           0.6543     0.5354  0.6543    0.5837
================================================================
```

DistilBERT-Multilingual achieved 73.4% accuracy, a weighted F1 of 0.721 and a macro F1 of 0.660. This is about 9.5 points higher in accuracy and 8 points higher in F1 than the best Logistic Regression model. On the English subset alone, DistilBERT reached 74.1% against 63.9% for Logistic Regression on the same 460 reviews, an improvement of about 10 points. Performance is consistent across languages: 71.4% for German, 74.1% for English and 74.6% for French.

```
================================================================
            SIDE-BY-SIDE MODEL PERFORMANCE COMPARISON
================================================================
                                        Accuracy  Precision  Recall  F1-Score
LogisticRegression                        0.6391     0.6413  0.6391    0.6398
LogisticRegression Hyperparameter Tuned   0.6391     0.6413  0.6391    0.6398
LinearSVC                                 0.6413     0.6205  0.6413    0.6275
MultinomialNB                             0.6543     0.5354  0.6543    0.5837
DistilBERT-Multilingual                   0.7340     0.7203  0.7340    0.7208
================================================================
=========================================================
           MODEL PERFORMANCE BREAKDOWN BY LANGUAGE
=========================================================
Language  Accuracy  Samples
      de  0.713996      493
      en  0.741304      460
      fr  0.745887      547
=========================================================
```

Most of DistilBERT's gain comes from the Positive and Negative classes, where it understands the context of the whole review. Neutral reviews remain difficult for every model. A likely cause is that Neutral labels come from 3-star ratings, which are noisy and often contain reviews that lean positive or negative.

```
DistilBERT metrics by sentiment:
              precision    recall  f1-score   support

    negative     0.7198    0.8792    0.7915       596
     neutral     0.4592    0.3082    0.3689       292
    positive     0.8455    0.7958    0.8199       612

    accuracy                         0.7340      1500
   macro avg     0.6748    0.6611    0.6601      1500
weighted avg     0.7203    0.7340    0.7208      1500
```

Conclusion and Limitations

Neutral sentiment is hard for classical ML

Predicting the Neutral class is difficult for the classical models. In the Logistic Regression results, the Neutral metrics are low (about 30%), while the other two classes score above 70%. The Neutral class therefore pulls down the overall performance of the classical models considerably. This is less of a problem for DistilBERT-Multilingual, which handles Neutral reviews somewhat

better because it understands the context of the whole review, although Neutral remains its weakest class.

Classical ML was limited to English data (~6,000 reviews)

The training set contains about 20,000 reviews, but only about 6,000 are in English. Our classical pipeline (stemming, lemmatization and stopword removal) is English-specific and does not support multiple languages out of the box, so we filtered the data to English only. We tried an open-source Google translation library, but it only supports about 200 KB of data per day, which was not enough for our dataset. We were therefore restricted to the English subset.

DistilBERT-Multilingual needs a GPU

On a CPU, training on the ~20,000 reviews was too slow, so we cancelled it after more than 4 hours. We moved the training to a Google Colab GPU (L4); with parallel processing and a batch size of 32, training took about 45 minutes. Training transformer models is time-consuming and needs more computational power, so computational capacity should be considered when choosing a model.

Labels based on star ratings

We assumed 1-2 stars = negative, 3 stars = neutral and 4-5 stars = positive. In reality, many 3-star reviews are mildly positive or negative, and some reviewers rate inconsistently with their text. This noise limits the maximum accuracy any model can reach, and it affects the Neutral class hardest, as seen in the evaluation.

Sentiment classes are imbalanced

Neutral reviews are fewer than Positive and Negative ones, so the models learn less from them. We used class_weight="balanced" to reduce this, but weights cannot replace real data.

Reviews mix product feedback with non-product issues

The star rating reflects the customer's whole experience, not just the product. Many reviews mention delivery delays, damaged packaging, shipping costs, wrong items, returns or customer service. A possible improvement is Aspect-Based Sentiment Analysis (ABSA), which labels sentiment separately for aspects such as product quality, delivery and customer service.

Final Recommendation

Based on the results, we recommend **DistilBERT-Multilingual** for the sentiment analysis task. It achieved the best overall performance, with an accuracy of 73.4% and an F1-score of 0.721. The best classical model, Logistic Regression, reached 63.9% accuracy and an F1-score of 0.640, so DistilBERT improves accuracy by about 10 percentage points and F1 by about 8 points. Multinomial Naive Bayes had a slightly higher accuracy (65.4%) than the other classical models, but its low precision (0.535) and F1-score (0.584) show that it favors the majority classes and performs poorly overall.

DistilBERT also performs consistently across all three languages: 71.4% for German, 74.1% for English and 74.6% for French. Hyperparameter tuning did not improve Logistic Regression, as the

tuned model gave the same scores as the baseline, which suggests the default settings were already close to optimal for this data.

The trade-off is cost and time. DistilBERT needs a GPU for training (about 45 minutes on a Colab L4). If speed and low cost matter more than accuracy, the Logistic Regression model is a reasonable alternative for English-only data.

References

Scikit-learn developers (n.d.) Getting Started. Available at: https://scikit-learn.org/stable/getting_started.html